Practical Viva

Practical 2

The McCulloch-Pitts (M-P) neuron is the grandfather of all neural networks. It is incredibly basic: it only accepts binary inputs (`0` or `1`), sums them up based on weights, and fires (`1`) only if that sum is greater than or equal to a specific threshold (θ).

The Logic of ANDNOT

An `ANDNOT` gate (A AND NOT B) has a very specific rule: **It only fires if Input 1 is ON and Input 2 is OFF.**

Here is the truth table:

- `0, 0` $\rightarrow$ `0`
- `0, 1` $\rightarrow$ `0`
- `1, 0` $\rightarrow$ `1` (The only time it fires)
- `1, 1` $\rightarrow$ `0`

How M-P Solves This

Your lab manual mentions **Excitatory** and **Inhibitory** inputs.

- **Excitatory (x_1):** Wants the neuron to fire. We give it a positive weight of `1`.
- **Inhibitory (x_2):** Wants to stop the neuron from firing. We give it a negative weight of -`1`.

If we set our threshold (θ) to `1`, the math perfectly matches the `ANDNOT` truth table.

The Python Code

Here is the exact code you need to execute this. It loops through all possible `(0,1)` combinations, applies the weights, checks against the threshold, and prints the result.

```python
def ANDNOT_logic_gate(x1, x2):
    # Set weights
    w1 = 1  # Excitatory
    w2 = -1 # Inhibitory
    threshold = 1

    # Aggregation function (g(x) from your manual)
    z = (x1 * w1) + (x2 * w2)

    # Decision function (f(x) from your manual)
    if z >= threshold:
        return 1
    else:
        return 0

# Test the network against all combinations
inputs = [(0, 0), (0, 1), (1, 0), (1, 1)]

print("x1 | x2 | Output")
print("----------------")
for x1, x2 in inputs:
    output = ANDNOT_logic_gate(x1, x2)
    print(f" {x1} |  {x2} |   {output}")
```

Viva Prep

- **Examiner:** "Why did you choose `1` for x_2?"
- **You:** "Because x_2 is an inhibitory input for the ANDNOT logic. If x_2 is 1, it must prevent the neuron from reaching the threshold of 1, regardless of x_1."

To really nail this for the viva, you should know how to change the weights and threshold to create other logic gates like `AND` or `OR`. Play with this M-P simulator to see how adjusting the threshold (θ) and the inhibitory/excitatory weights instantly changes the logic gate.

Practical 3

```
# Import libraries
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score, confusion_matri
x
import matplotlib.pyplot as plt

# Load dataset
digits = load_digits()

# Features and labels
X = digits.data
y = digits.target

# Convert to even (0) and odd (1)
y = np.where(y % 2 == 0, 0, 1)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train perceptron model
model = Perceptron(max_iter=1000)
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

# Confusion Matrix
```

```
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pre
d))

# Show some predictions
for i in range(5):
    plt.imshow(X_test[i].reshape(8,8), cmap='gray')
    plt.title(f"Actual: {y_test[i]}  Predicted: {y_pred
[i]}")
    plt.show()
```

1. Loading the Data

Python

```
digits = load_digits()
X = digits.data
y = digits.target
```

- **What it does:** It downloads a famous dataset of handwritten digits.
- **The Features (`X`):** These are the actual images. Each image is 8×8 pixels. However, the Perceptron can't read a 2D grid, so the dataset flattens each image into a single 1D array of **64 numbers** (representing pixel brightness).
- **The Labels (`y`):** These are the correct answers (e.g., "this image is a 3", "this image is a 4").

2. The Core Hack (Binary Conversion)

Python

```
y = np.where(y % 2 == 0, 0, 1)
```

- **What it does:** A standard Perceptron is a **binary classifier**. It cannot sort things into 10 different buckets (0 through 9); it can only answer "Yes" or "No".
- **How it works:** This line overwrites the answers in `y`. It divides every answer by 2. If the remainder is 0, it labels the image `0` (Even). Otherwise, it labels it `1` (Odd). It successfully turns a 10-class problem into a 2-class problem.

3. Splitting the Data

Python

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

- **What it does:** You never test a model on the same data you used to teach it, otherwise it just memorizes the answers. This takes 80% of the images to train the Perceptron (`X_train`) and hides the remaining 20% to test it later as a final exam (`X_test`).

You can set `random_state=0`, `random_state=1`, or `random_state=9999`. They all work perfectly fine to lock the seed.

The number 42 is an inside joke in the programming community. It comes from the famous sci-fi book *The Hitchhiker's Guide to the Galaxy* by Douglas Adams. In the book, a supercomputer calculates "The Answer to the Ultimate Question of Life, the Universe, and Everything"—and after millions of years of calculating, the answer it spits out is simply **42**.

Because data scientists are trying to find "answers" in data, they started using 42 as the default random seed. It became a global convention.

4. Training the Brain

Python

```
model = Perceptron(max_iter=1000)
model.fit(X_train, y_train)
```

- **What it does:** It creates the Perceptron. `max_iter=1000` means it will loop through the training images a maximum of 1,000 times.
- **`model.fit`:** This is where the actual learning happens. The Perceptron looks at the 64 pixels of an image, guesses if it's even or odd, checks the real answer in `y_train`, and updates its 64 internal weights if it guessed wrong.

5. The Final Exam & Grading

Python

```
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

- **What it does:** It passes the 20% hidden images (`X_test`) to the model and asks for predictions (`y_pred`).
- **Accuracy:** It compares the model's guesses against the actual hidden answers (`y_test`) and prints a percentage (e.g., 0.85 or 85%).

- **Confusion Matrix:** It prints a 2×2 table showing exactly *how* it failed (e.g., how many odd numbers it accidentally called even).

6. Visualizing the Output

Python

```
for i in range(5):
    plt.imshow(X_test[i].reshape(8,8), cmap='gray')
    plt.title(f"Actual: {y_test[i]}  Predicted: {y_pred[i]}")
    plt.show()
```

- **What it does:** It loops through the first 5 test images to show you the results visually.
- `reshape(8,8)`: Remember how the images were flattened into an array of 64 numbers at the start? `matplotlib` cannot draw a flat line. This line folds those 64 numbers back into an 8×8 pixel grid so it can render a recognizable picture on your screen.

Practical 5

```
# Recognize digits 0 to 9 using 5x3 matrix and Neural Netwo
rk

import numpy as np
from sklearn.neural_network import MLPClassifier

# --------------------------------
# Step 1: Define 5x3 patterns for digits 0 to 9
# 1 = valid point
# 0 = invalid point
# Total inputs = 15 (5 rows × 3 cols)
# --------------------------------

digits = {
    0: [1,1,1,
        1,0,1,
        1,0,1,
        1,0,1,
        1,1,1],
```

```
1: [0,1,0,
    1,1,0,
    0,1,0,
    0,1,0,
    1,1,1],

2: [1,1,1,
    0,0,1,
    1,1,1,
    1,0,0,
    1,1,1],

3: [1,1,1,
    0,0,1,
    1,1,1,
    0,0,1,
    1,1,1],

4: [1,0,1,
    1,0,1,
    1,1,1,
    0,0,1,
    0,0,1],

5: [1,1,1,
    1,0,0,
    1,1,1,
    0,0,1,
    1,1,1],

6: [1,1,1,
    1,0,0,
    1,1,1,
    1,0,1,
    1,1,1],

7: [1,1,1,
```

```
        0,0,1,
        0,1,0,
        0,1,0,
        0,1,0],

    8: [1,1,1,
        1,0,1,
        1,1,1,
        1,0,1,
        1,1,1],

    9: [1,1,1,
        1,0,1,
        1,1,1,
        0,0,1,
        1,1,1]
}

# --------------------------------
# Step 2: Prepare training data
# --------------------------------

X = []
y = []

for num in digits:
    X.append(digits[num])
    y.append(num)

X = np.array(X)
y = np.array(y)

# --------------------------------
# Step 3: Create Neural Network
# hidden_layer_sizes=(10,) means 1 hidden layer with 10 neu
rons
# max_iter=5000 means training iterations
# --------------------------------
```

```
model = MLPClassifier(hidden_layer_sizes=(10,),
                      max_iter=5000,
                      random_state=1)

# --------------------------------
# Step 4: Train the network
# --------------------------------

model.fit(X, y)

# --------------------------------
# Step 5: Test with sample input
# Change pattern to test another digit
# --------------------------------

test = [[1,1,1,
         0,0,1,
         1,1,1,
         0,0,1,
         1,1,1]]   # This is digit 3

prediction = model.predict(test)

print("Recognized Number =", prediction[0])
```

The Core Problem: The 5x3 Matrix

You are teaching a neural network to read digital numbers, similar to an old digital alarm clock. Every number from 0 to 9 can be drawn on a tiny grid that is 5 rows tall and 3 columns wide.

- **Valid Point (1):** The pixel is "drawn" (colored in).
- **Invalid Point (0):** The pixel is blank.

The Engineering Trick: Flattening

A standard Artificial Neural Network (ANN) cannot process a 2D grid. The math ($Z = X \cdot W + b$) requires a 1D vector (a single flat line of numbers).

Your Viva Defense

- **Examiner:** "Why did you use 15 input nodes?"
  - **You:** "Because the problem specifies a 5×3 matrix. To feed a 2D matrix into a standard feedforward neural network, it must be flattened into a 1D vector. 5 multiplied by 3 gives us a 15-dimensional input vector."
- **Examiner:** "What is the shape of your weight matrix between the input and the hidden layer?"
  - **You:** "If my input layer has 15 nodes, and my hidden layer has 15 nodes, the weight matrix connecting them is a 15×15 matrix (containing 225 individual weights). This aligns with the math in the lab manual."

`MLPClassifier` stands for **Multi-Layer Perceptron Classifier**. It is the standard, fully-connected, feedforward neural network you studied in the theory for Practical 5.

Here is the exact breakdown of the three parameters you passed into it, and how to defend them in your viva.

1. `hidden_layer_sizes=(10,)`

This parameter dictates the exact physical architecture of your hidden layers.

- **The Number `10`:** This means you are creating exactly **10 neurons** (or nodes) in your hidden layer.
- **The Comma `(10,)`:** This is a classic Python "gotcha" and a favorite viva question. In Python, adding a comma inside parentheses creates a **Tuple**.
  - Because there is only one number inside the tuple, it means you have exactly **1 hidden layer** with 10 nodes.
  - If you wanted *two* hidden layers (the first with 10 nodes, the second with 5), you would write `hidden_layer_sizes=(10, 5)`.

2. `max_iter=5000`

This is your training time limit. "Iter" stands for iterations (often called epochs).

- **What it does:** It tells the backpropagation algorithm, *"You are allowed to loop through the training data and adjust your weights a maximum of 5,000*

times."

- **Viva Point:** The network might not actually use all 5,000 iterations. If the network reaches a point where the error (loss) stops shrinking—meaning it has learned as much as it possibly can—it will automatically stop early to save computing power. `5000` is simply a hard ceiling to prevent the algorithm from getting stuck in an infinite loop.

3. `random_state=1`

We touched on this when discussing the `train_test_split` logic, but it does something slightly different here inside the actual neural network.

- **What it does:** When an MLP initializes, its starting weights and biases are completely random numbers. `random_state=1` locks the random number generator's seed.
- **Viva Point:** By setting this to `1` (or any constant number), you guarantee that the network will initialize with the **exact same starting weights** every single time you run the script. Without this, your accuracy would fluctuate by a few percent every time you click "Run," making it impossible to tell if your hyperparameter tuning actually improved the model or if you just got a "lucky" random initialization.

Practical 6

You are completely right. Using your "from scratch" code is actually the much better move for this specific practical. Since the title is *"To illustrate perceptron learning law,"* an examiner will be much happier seeing you write the actual math manually rather than hiding it behind an imported library.

Here is the single, unified breakdown of exactly what this practical requires and how your code perfectly executes it.

Part 1: What this Practical is Asking You to Do

This assignment tests two specific concepts from your theory manual:

1. **The Perceptron Learning Law:** You need to prove you know how a neural network "learns." It learns by making a guess, checking if it was wrong, and mathematically tweaking its weights and bias to do better next time.

2. **The Decision Regions:** You need to prove that a Perceptron is a "linear classifier." You have to draw a graph showing a straight line that slices a 2D space into two distinct territories (one region for Class 1, and another region for Class -1).

Part 2: How Your Code Works (Block-by-Block)

1. The Setup (Creating the Universe)

Python

```
X = np.array([[1, 1], [2, 1], [4, 3], [5, 4]])
y = np.array([-1, -1, 1, 1])
w = np.zeros(2)
b = 0
lr = 1
```

- **The Data (`X` and `y`):** You created four coordinates on a graph. The first two points belong to the Blue team (`1`), and the last two belong to the Red team (`1`).
- **The Brain (`w`, `b`, `lr`):** You initialize two weights at `0` (because your points have an X and Y coordinate). You set the bias to `0`. Your Learning Rate (`lr`) is `1`, meaning the network takes full mathematical steps when correcting its mistakes.

2. The Perceptron Learning Law (The Engine)

Python

```
for epoch in range(10):
    for i in range(len(X)):
        net = np.dot(X[i], w) + b

        if net >= 0: output = 1
        else: output = -1

        if output != y[i]:
            w = w + lr * y[i] * X[i]
            b = b + lr * y[i]
```

This is the heart of the practical. Here is what happens inside that loop:

- **The Guess (`net`):** It multiplies the input by the weights and adds the bias. If the result is positive, it guesses Red (`1`). If negative, it guesses Blue (`1`).
- **The Correction (`if output != y[i]`):** If the Perceptron guesses wrong, it fires the **Learning Rule**. It updates the weights using the exact formula from your manual: $W_{new} = W_{old} + (\text{Learning Rate} \times \text{Target Output} \times \text{Input})$. It does this 10 times (`epochs`) until it finds the perfect weights to separate the red and blue points.

3. Creating the Decision Regions (The Grid)

Python

```
xx, yy = np.meshgrid(np.arange(x1_min, x1_max, 0.1),
                     np.arange(x2_min, x2_max, 0.1))
```

- **What it does:** To color the background of a graph, you can't just draw a shape. You have to create thousands of invisible, tiny coordinate points that cover the entire screen. `meshgrid` generates this invisible background grid.

4. Classifying the Background

Python

```
for i in range(len(xx.ravel())):
    point = np.array([xx.ravel()[i], yy.ravel()[i]])
    net = np.dot(point, w) + b
    if net >= 0: Z.append(1)
    else: Z.append(-1)
```

- **What it does:** This loop takes your fully trained weights and applies them to every single one of those tiny invisible background points (`xx.ravel()`). It forces the Perceptron to predict if that specific pixel belongs to the Red team (`1`) or the Blue team (`1`), and saves that answer in the `Z` array.

5. Drawing the Output

Python

```
plt.contourf(xx, yy, Z, alpha=0.4, cmap="bwr")
plt.scatter(X[:,0], X[:,1], c=y, cmap="bwr", s=100, edgecolors="black")
```

- `contourf`: This looks at your `Z` array and colors the background grid based on the Perceptron's predictions. This creates the visual "Decision Regions" (Blue vs. Red background).
- `scatter`: This drops your original 4 data points exactly where they belong on top of the colored regions so the examiner can see that the Perceptron successfully separated them.

Your Viva Defense for this Code

If the examiner asks why you wrote it this way instead of using a library, simply say:

> *"I wanted to manually implement the Perceptron Learning Rule to demonstrate how the weight update formula physically shifts the decision boundary over 10 epochs. By writing the `meshgrid` classification manually, it clearly illustrates how the linear equation $W |cdot X + b = 0$ dictates the colored decision regions on the graph."*

Practical 7

The Core Problem: Why XOR?

The practical explicitly asks you to solve the **XOR function**. Why XOR? Because it is a trap for basic AI.

If you graph the XOR truth table, you get two "True" points (1,0 and 0,1) and two "False" points (0,0 and 1,1) positioned diagonally from each other.

You cannot draw a single straight line to separate the True points from the False points.

Because it is not "linearly separable," a basic single-layer Perceptron (like the one you used for Practical 3) will completely fail to solve XOR. To solve it, you *must* add a **Hidden Layer**, and to train a hidden layer, you *must* use **Backpropagation**.

How Your Code Works (Block-by-Block)

1. The Setup & Architecture

Python

```
W1 = np.random.rand(2, 2) # Input to Hidden
W2 = np.random.rand(2, 1) # Hidden to Output
```

- **What it does:** You are building a network with 2 Input Nodes, 2 Hidden Nodes, and 1 Output Node.
- **The Shapes:**
  - `W1` is a 2×2 matrix because 2 inputs are connecting to 2 hidden nodes (4 total connections).
  - `W2` is a 2×1 matrix because 2 hidden nodes are connecting to 1 final output node (2 connections).

2. Forward Propagation (The Guess)

Python

```
H = sigmoid(np.dot(X, W1) + b1)
O = sigmoid(np.dot(H, W2) + b2)
```

- **What it does:** Data flows strictly left-to-right.
- It multiplies the inputs (`X`) by the first set of weights, adds the bias, and squashes it through the Sigmoid function to get the Hidden Layer's values (`H`).
- Then, it does the exact same thing again, using `H` as the new input, to get the final Output guess (`O`).

3. Back Propagation (The Calculus)

Python

```
error = Y - O
dO = error * sigmoid_derivative(O)
dH = np.dot(dO, W2.T) * sigmoid_derivative(H)
```

- **What it does:** The data flows right-to-left.
- It finds out how wrong the final answer is (`error`).
- `dO` (Delta Output): Uses the derivative of Sigmoid to calculate the gradient (the slope of the error). This tells the network how to fix `W2`.

- `dH` (Delta Hidden): It pushes that error backward across `W2` to figure out how much the Hidden Layer was responsible for the mistake. It uses the derivative again to know how to fix `W1`.

1. What is `.T` (The Transpose)?
In Python's `numpy`, adding `.T` to the end of a matrix completely flips it over its diagonal. Rows become columns, and columns become rows. If you have a matrix that is 2 rows by 3 columns (2×3), calling `.T` turns it into 3×2.
Why do we need it in Backpropagation?
It is strictly required by the rules of Matrix Multiplication (the Dot Product).
For a computer to multiply two matrices together, the inner dimensions **must match**. You can multiply a $(4 \times \mathbf{2})$ matrix by a $(\mathbf{2} \times 1)$ matrix, but you cannot multiply a $(4 \times \mathbf{1})$ by a $(\mathbf{2} \times 1)$.
Look at this line from your code:
`dH = np.dot(dO, W2.T)`
- `dO` **(Output Error):** Contains 4 errors (one for each XOR row). Its shape is (4×1).
- `W2` **(Weights):** Connects 2 hidden nodes to 1 output node. Its shape is (2×1).

If you try `np.dot(dO, W2)`, Python will crash because the dimensions $(4 \times \mathbf{1})$ and $(\mathbf{2} \times 1)$ don't match.
By calling `W2.T`, you flip the weights to a (1×2) shape. Now, $(4 \times \mathbf{1}) \cdot (\mathbf{1} \times 2)$ matches perfectly.
Viva Answer: *"The `.T` transposes the matrix. During backpropagation, we have to route the error backwards through the network. Transposing the weight matrix aligns the dimensions so the Dot Product can successfully map the output error back to the specific hidden nodes that caused it."*

4. Updating the Network (Gradient Descent)

Python

```
W2 = W2 + np.dot(H.T, dO) * 0.1
W1 = W1 + np.dot(X.T, dH) * 0.1
```

- **What it does:** It applies the fixes. It calculates the new weights by taking the old weights and adding the gradient multiplied by `0.1`.
- **The Learning Rate:** That `0.1` is your learning rate. It prevents the network from overcorrecting and violently swinging its weights back and forth.

5. The Output

Python

`print(O.round())`

- **What it does:** The Sigmoid function outputs probabilities (like 0.98 or 0.03). Because the practical asks for "Binary Output," using `.round()` snaps those probabilities to a clean `1.0` or `0.0`.

Your Viva Defense

- **Examiner:** "Why did you use `.T` (Transpose) on the matrices during the weight update, like `H.T` and `X.T`?"
  - **You:** "Because of the rules of matrix multiplication (Dot Product). When pushing the error backwards, the dimensions of the activation matrices and the error gradients do not align naturally. Transposing the matrix flips its rows and columns, ensuring the error gets mathematically routed to the exact specific weight that caused it."
- **Examiner:** "Why did you use `.sum(axis=0)` when updating the biases?"
  - **You:** "During backpropagation, the network processes all 4 rows of the XOR truth table simultaneously. The error gradient (`dH`) is a matrix containing the errors for all 4 examples. By using `sum(axis=0)`, I am collapsing those 4 separate errors into a single, averaged adjustment to correctly update the bias for the entire batch."

This code is incredibly clean and does exactly what the manual asks. Let me know when you are ready to jump into the final advanced algorithms from Group B (ART or Hopfield)!

Practical 8

The Core Problem: The Stability-Plasticity Dilemma

Standard neural networks have "catastrophic forgetting." If you train a network to recognize dogs, and then later try to teach it to recognize cats, it alters its

weights and completely forgets what a dog looks like.

- **Plasticity:** The ability to learn new things.
- **Stability:** The ability to remember old things.

ART solves this dilemma. If it sees something familiar, it refines its memory (**Resonance/Stability**). If it sees something completely brand new, it doesn't overwrite its old memories; it creates a brand new memory cluster (**Adaptive/Plasticity**).

1. The Dataset & Parameters

```
import numpy as np

X = np.array([
    [1, 1, 0, 0],
    [1, 1, 0, 1],
    [0, 0, 1, 1],
    [0, 1, 1, 1]
])

num_features = 4
max_clusters = 4
vigilance = 0.6
```

- `X` **(The Input Data):** You are feeding the network 4 distinct binary patterns. Because each row has 4 numbers, your data is 4-dimensional.
- `num_features = 4`: Tells the network to expect 4 inputs per pattern.
- `max_clusters = 4`: Sets a hard limit on the network. It is only allowed to create a maximum of 4 memory categories.
- `vigilance = 0.6`: This is the strictness threshold. For a new pattern to join an existing cluster, it must mathematically match the cluster's memory template by at least 60%.

2. Initializing the Two Sets of Weights

Python

```
W = np.ones((max_clusters, num_features)) / (1 + num_featur
es)
T = np.ones((max_clusters, num_features))
clusters_used = 0
```

ART1 requires two weight matrices to function (The Two-Thirds Rule).

- `W` **(Bottom-Up Weights):** This is the network's "search engine" used to calculate the matching score. `np.ones(...) / (1 + 4)` fills this entire matrix with the fraction `0.2`. Starting with small fractions ensures blank clusters don't accidentally win the matching competition.
- `T` **(Top-Down Weights):** This is the network's actual "Memory." `np.ones(...)` fills this matrix entirely with `1`s. Initializing with `1`s means every cluster starts as a blank slate, fully open to inheriting the exact binary pattern of the first input that joins it.
- `clusters_used = 0`: A counter to keep track of how many of our 4 `max_clusters` have actually been built so far.

3. The Search & The Vigilance Test

Python

```
for pattern in X:
    assigned = False

    for j in range(clusters_used):
        match = np.sum(pattern * T[j]) / np.sum(pattern)
```

- `for pattern in X:` The network grabs one pattern from the dataset at a time.
- `assigned = False`: A flag to track if the pattern has found a home yet.
- `for j in range(clusters_used):` The network loops through the clusters it has already created to see if the new pattern belongs to any of them.
- `match = ...`: This calculates the percentage match.
  - `pattern * T[j]` does a logical AND operation (e.g., `1 * 1 = 1`, `1 * 0 = 0`). It counts exactly how many `1`s overlap perfectly between the input

pattern and the memory template.

- Dividing it by `np.sum(pattern)` (the total number of `1`s in the input) gives the final match percentage (like `0.75`).

4. Resonance (Joining an Existing Cluster)

Python

```
        if match >= vigilance:
            T[j] = pattern * T[j]
            W[j] = T[j] / (0.5 + np.sum(T[j]))
            print("Pattern", pattern, "belongs to Cluster",
j + 1)
            assigned = True
            break
```

If the `match` is 0.6 or higher, the network accepts the pattern into cluster `j`.

- `T[j] = pattern * T[j]`: The network updates the memory template using a logical AND. This means the memory template *only* keeps a `1` if both the old memory and the new pattern share it. The memory shrinks down to only the core features shared by all members of the cluster.
- `W[j] = T[j] / (0.5 + np.sum(T[j]))`: The network updates the search engine weights. It divides the new memory by the total number of `1`s in the memory. (The `0.5` is a small tie-breaker value to prevent division by zero errors and balance the competition).
- `assigned = True` & `break`: The pattern has found a home, so it flips the flag to `True` and breaks out of the search loop to move on to the next input pattern.

5. Plasticity (Creating a New Cluster)

Python

```
    if not assigned:
        T[clusters_used] = pattern
        W[clusters_used] = pattern / (0.5 + np.sum(patter
n))
        print("Pattern", pattern, "creates Cluster", cluste
```

```
rs_used + 1)
        clusters_used += 1
```

If the pattern failed the vigilance test for *every single existing cluster*, the `assigned` flag remains `False`. The network triggers a "reset."

- `T[clusters_used] = pattern`: Instead of overwriting an old memory, the network selects a completely blank cluster and permanently burns the exact input pattern into it as a brand-new memory template.
- `W[clusters_used] = ...`: It initializes the bottom-up search weights for this new cluster based on the input pattern.
- `clusters_used += 1`: It updates the counter so the network knows it has one more active cluster to search through the next time a pattern comes in.

6. The Final Output

Python

```python
print("\nFinal Top-Down Weights:")
for i in range(clusters_used):
    print("Cluster", i + 1, ":", T[i])
```

- Once all 4 patterns have been processed, this simple loop prints out the final `T` matrix.
- Because `T` is the network's Memory Template, printing `T[i]` physically shows you the final binary shape of each cluster it created, allowing you to see exactly which features it grouped together.

Practical 9

You have officially made it to the final practical!

This assignment introduces the **Hopfield Network**, which operates completely differently from everything else you've built. Perceptrons and MLPs are built for *classification* (guessing an answer). Hopfield Networks are built for **Memory Recall**.

Here is the straightforward breakdown of the theory and a line-by-line explanation of your code.

The Core Concept: Auto-Associative Memory

Think about how human memory works. If you hear the first 3 seconds of a song you know, your brain instantly fills in the rest of the melody. You don't need the whole song to recognize it; a "noisy" or partial piece of information is enough to trigger the complete memory.

A Hopfield Network does exactly this mathematically. It is an **Auto-Associative Memory** network.

1. You store complete patterns (vectors) inside its weights.
2. You feed the network a corrupted, broken, or "noisy" version of a pattern.
3. The network uses feedback loops to update itself until it "snaps" back into the original, clean pattern.

How Your Code Works (Block-by-Block)

1. The Memories (Bipolar Patterns)

Python

```
patterns = np.array([
    [1,  1,  1, -1],
    [1, -1,  1, -1],
    [-1, 1, -1, 1],
    [-1,-1, 1, 1]
])
n = patterns.shape[1]
```

- **What it does:** You define 4 distinct memories you want the network to learn.
- Notice that instead of `1` and `0`, the network uses `1` and `1`. This is called **Bipolar Data**. Hopfield networks are mathematically much more stable when using bipolar values because `1` actively suppresses neurons, whereas `0` just does nothing.
- `n = 4` because each vector has exactly 4 elements (representing 4 neurons in the network).

2. Storing the Memories (Hebbian Learning)

Python

```
W = np.zeros((n, n))

for p in patterns:
    W = W + np.outer(p, p)   # Outer product

np.fill_diagonal(W, 0)
```

This is where the network "memorizes" the data. Hopfield networks do not use Backpropagation. They use **Hebbian Learning**, which follows a famous rule in neuroscience: *"Neurons that fire together, wire together."*

- **`np.outer(p, p)`:** This takes a 1D vector and multiplies it by itself to create a 2D matrix. It strengthens the connection between neurons if they have the same sign (both `1` or both `1`), and weakens it if they have opposite signs. It adds this result to the master weight matrix `W`.
- **`np.fill_diagonal(W, 0)`:** This is a crucial rule of Hopfield architecture. A neuron is connected to every *other* neuron, but it is **not connected to itself**. The diagonal of the weight matrix represents self-connections, so we forcefully set them to `0`.

3. The Corrupted Memory (Testing)

Python

```
test = np.array([1, 1, -1, -1])
```

- **What it does:** You introduce a new pattern that is not in your original memory list. It is a "noisy" input. The network needs to figure out which of its 4 stored memories this noisy input most closely resembles.

4. The Recall Process (The Feedback Loop)

Python

```
for i in range(5):
    test = np.sign(np.dot(W, test))
    print("Iteration", i+1, ":", test)
```

- **What it does:** This is the recall mechanism. The network multiplies the noisy input by the memory weights (`np.dot`).
- **`np.sign()`:** This is the activation function. If the math results in a positive number, `np.sign` snaps it to `1`. If negative, it snaps it to `1`.

- Because Hopfield networks are **recurrent** (they have feedback loops), you take the output and feed it right back in as the new input. After a few iterations, the state stops changing—the network has "settled" into a stable state. It has successfully recovered the memory.

Your Viva Defense

Because this network is structurally unique, examiners will focus on how it differs from a standard feedforward network.

- **Examiner:** "Why did you use `np.outer(p, p)` to calculate the weights?"
  - **You:** "Hopfield networks use Hebbian learning rather than gradient descent. The outer product calculates the cross-correlation between every pair of neurons in the pattern. It mathematically encodes the rule that if two neurons share the same state, their connection weight increases, embedding the pattern into the weight matrix."
- **Examiner:** "Why is it necessary to set the diagonal of the weight matrix to zero?"
  - **You:** "The diagonal represents a neuron's connection to itself. In a discrete Hopfield network, self-feedback is not allowed. If a neuron fed its own output directly back into itself, it would create an infinite feedback loop, and the network would never converge to a stable memory state."
- **Examiner:** "What is the purpose of `np.sign()` during the recall phase?"
  - **You:** "It acts as a hard-limit activation function. Because we are using Bipolar data, the dot product of the weights and inputs will result in continuous numbers like 4.2 or -3.1. `np.sign()` instantly snaps those values back into discrete `1`s or `1`s, updating the state of the neurons for the next iteration."

To help visualize how a noisy input mathematically settles into a clean memory over a few iterations, you can play around with this interactive Hopfield simulator.